

IIC2513 – Tecnologías y Aplicaciones Web

Ejercicios I2

Semestre
1 • 2026

Profesor
Hernán Cabrera

Ayudante
Arturo Pacheco

[EJ.01](#) • Enunciado

Explique brevemente la estructura de un `JWT` y qué función cumple cada una de sus partes.

Explique qué son los **rainbow tables** y cómo `bcrypt` los previene.

[— SOLUCIÓN](#)

EJ.01 • Enunciado

Explique brevemente la estructura de un `JWT` y qué función cumple cada una de sus partes.

Explique qué son los **rainbow tables** y cómo `bcrypt` los previene.

— SOLUCIÓN



header

.

payload

.

signature

EJ.01 · Enunciado

Explique brevemente la estructura de un `JWT` y qué función cumple cada una de sus partes.

Explique qué son los **rainbow tables** y cómo `bcrypt` los previene.

— SOLUCIÓN

header

.

payload

.

signature

- 01 **Header** — tipo de token (JWT) y algoritmo de firma (HS256, RS256...).

EJ.01 · Enunciado

Explique brevemente la estructura de un `JWT` y qué función cumple cada una de sus partes.

Explique qué son los **rainbow tables** y cómo `bcrypt` los previene.

— SOLUCIÓN

header

.

payload

.

signature

01 **Header** — tipo de token (JWT) y algoritmo de firma (HS256, RS256...).

02 **Payload** — claims/datos del usuario, p.ej. `{ id: 123, role: "admin" }`.

EJ.01 · Enunciado

Explique brevemente la estructura de un `JWT` y qué función cumple cada una de sus partes.

Explique qué son los **rainbow tables** y cómo `bcrypt` los previene.

— SOLUCIÓN

header

.

payload

.

signature

- 01 **Header** — tipo de token (JWT) y algoritmo de firma (HS256, RS256...).
- 02 **Payload** — claims/datos del usuario, p.ej. `{ id: 123, role: "admin" }`.
- 03 **Signature** — header+payload firmados con una clave secreta. Permite verificar que el token no fue modificado.

EJ.01 · Enunciado

Explique brevemente la estructura de un `JWT` y qué función cumple cada una de sus partes.

Explique qué son los **rainbow tables** y cómo `bcrypt` los previene.

— SOLUCIÓN

header

.

payload

.

signature

- 01 **Header** — tipo de token (JWT) y algoritmo de firma (HS256, RS256...).
- 02 **Payload** — claims/datos del usuario, p.ej. `{ id: 123, role: "admin" }`.
- 03 **Signature** — header+payload firmados con una clave secreta. Permite verificar que el token no fue modificado.
- RT **Rainbow tables** — tablas precalculadas de hashes que se comparan contra los hashes almacenados para descubrir contraseñas.

EJ.01 · Enunciado

Explique brevemente la estructura de un `JWT` y qué función cumple cada una de sus partes.

Explique qué son los **rainbow tables** y cómo `bcrypt` los previene.

— SOLUCIÓN

header

.

payload

.

signature

- 01 **Header** — tipo de token (JWT) y algoritmo de firma (HS256, RS256...).
- 02 **Payload** — claims/datos del usuario, p.ej. `{ id: 123, role: "admin" }`.
- 03 **Signature** — header+payload firmados con una clave secreta. Permite verificar que el token no fue modificado.
- RT **Rainbow tables** — tablas precalculadas de hashes que se comparan contra los hashes almacenados para descubrir contraseñas.
- b1 **bcrypt · salt aleatorio** — agrega datos aleatorios distintos a cada contraseña antes de hashearla.

EJ.01 · Enunciado

Explique brevemente la estructura de un `JWT` y qué función cumple cada una de sus partes.

Explique qué son los **rainbow tables** y cómo `bcrypt` los previene.

— SOLUCIÓN

header

.

payload

.

signature

- 01 **Header** — tipo de token (JWT) y algoritmo de firma (HS256, RS256...).
- 02 **Payload** — claims/datos del usuario, p.ej. `{ id: 123, role: "admin" }`.
- 03 **Signature** — header+payload firmados con una clave secreta. Permite verificar que el token no fue modificado.
- RT **Rainbow tables** — tablas precalculadas de hashes que se comparan contra los hashes almacenados para descubrir contraseñas.
- b1 **bcrypt · salt aleatorio** — agrega datos aleatorios distintos a cada contraseña antes de hashearla.
- b2 **bcrypt · costo computacional** — es intencionalmente lento, dificultando ataques masivos.

[EJ.02](#) • Enunciado

Clasifique los códigos HTTP según su categoría (2xx, 3xx, 4xx, 5xx).

[— SOLUCIÓN](#)

[EJ.02](#) • Enunciado

Clasifique los códigos HTTP según su categoría (2xx, 3xx, 4xx, 5xx).

[— SOLUCIÓN](#)

2xx

Exitosas

200 OK

201 Created

[EJ.02](#) • Enunciado

Clasifique los códigos HTTP según su categoría (2xx, 3xx, 4xx, 5xx).

[— SOLUCIÓN](#)

2xx

Exitosas

200 OK

201 Created

3xx

Redirecciones

301 Moved Permanently

302 Found

EJ.02 • Enunciado

Clasifique los códigos HTTP según su categoría (2xx, 3xx, 4xx, 5xx).

SOLUCIÓN

2xx

Exitosas

200 OK

201 Created

3xx

Redirecciones

301 Moved Permanently

302 Found

4xx

Error del cliente

400 Bad Request

401 Unauthorized

404 Not Found

EJ.02 • Enunciado

Clasifique los códigos HTTP según su categoría (2xx, 3xx, 4xx, 5xx).

SOLUCIÓN

2xx

Exitosas

200 OK

201 Created

3xx

Redirecciones

301 Moved Permanently

302 Found

4xx

Error del cliente

400 Bad Request

401 Unauthorized

404 Not Found

5xx

Error del servidor

500 Internal Server
Error

503 Service Unavailable

EJ.03 • Enunciado

¿Cuál es la diferencia entre `Promise.all()` y `Promise.allSettled()`? ¿Cuándo usar una u otra?

— SOLUCIÓN —

EJ.03 • Enunciado

¿Cuál es la diferencia entre `Promise.all()` y `Promise.allSettled()`? ¿Cuándo usar una u otra?

— SOLUCIÓN

FALLA RÁPIDO

`Promise.all()`

Espera que **todas** las promesas se resuelvan exitosamente. Si una falla, toda la operación es **rechazada inmediatamente**.

```
Promise.all([p1, p2, p3])
```


EJ.03 • Enunciado

¿Cuál es la diferencia entre `Promise.all()` y `Promise.allSettled()`? ¿Cuándo usar una u otra?

SOLUCIÓN

FALLA RÁPIDO

`Promise.all()`

Espera que **todas** las promesas se resuelvan exitosamente. Si una falla, toda la operación es **rechazada inmediatamente**.

```
Promise.all([p1, p2, p3])
```

ESPERA A TODAS

`Promise.allSettled()`

Espera que **todas** terminen, sin importar si fueron resueltas o rechazadas. Entrega el resultado individual de cada una.

```
Promise.allSettled([p1, p2, p3])
```

EJ.03 • Enunciado

¿Cuál es la diferencia entre `Promise.all()` y `Promise.allSettled()`? ¿Cuándo usar una u otra?

— SOLUCIÓN —

FALLA RÁPIDO

`Promise.all()`

Espera que **todas** las promesas se resuelvan exitosamente. Si una falla, toda la operación es **rechazada inmediatamente**.

```
Promise.all([p1, p2, p3])
```

ESPERA A TODAS

`Promise.allSettled()`

Espera que **todas** terminen, sin importar si fueron resueltas o rechazadas. Entrega el resultado individual de cada una.

```
Promise.allSettled([p1, p2, p3])
```

- ✓ Usa `all()` cuando *todas* las operaciones son necesarias para continuar.

EJ.03 • Enunciado

¿Cuál es la diferencia entre `Promise.all()` y `Promise.allSettled()`? ¿Cuándo usar una u otra?

— SOLUCIÓN —

FALLA RÁPIDO

`Promise.all()`

Espera que **todas** las promesas se resuelvan exitosamente. Si una falla, toda la operación es **rechazada inmediatamente**.

```
Promise.all([p1, p2, p3])
```

- ✓ Usa `all()` cuando *todas* las operaciones son necesarias para continuar.

ESPERA A TODAS

`Promise.allSettled()`

Espera que **todas** terminen, sin importar si fueron resueltas o rechazadas. Entrega el resultado individual de cada una.

```
Promise.allSettled([p1, p2, p3])
```

- ✓ Usa `allSettled()` cuando interesa el resultado de todas, aunque algunas fallen.

[EJ.04](#) · Enunciado

Flujo completo: `POST /api/users` → `201 Created`

Considere cabeceras, validación, middleware de parseo y formato de respuesta.

[— SOLUCIÓN](#) · [PASO A PASO](#)

[EJ.04](#) · Enunciado

Flujo completo: `POST /api/users` → `201 Created`

Considere cabeceras, validación, middleware de parseo y formato de respuesta.

[— SOLUCIÓN](#) · [PASO A PASO](#)

1 El cliente envía `POST /api/users` con headers

```
Content-Type: application/json
Accept:      application/json
```


EJ.04 · Enunciado

Flujo completo: `POST /api/users` → `201 Created`

Considere cabeceras, validación, middleware de parseo y formato de respuesta.

[— SOLUCIÓN · PASO A PASO](#)

1 El cliente envía `POST /api/users` con headers

```
Content-Type: application/json
Accept:      application/json
```

2 Cuerpo en JSON

```
{
  "name": "Ana",
  "email": "ana@mail.com"
}
```

EJ.04 · Enunciado

Flujo completo: `POST /api/users` → `201 Created`

Considere cabeceras, validación, middleware de parseo y formato de respuesta.

[— SOLUCIÓN · PASO A PASO](#)

1 El cliente envía `POST /api/users` con headers

```
Content-Type: application/json
Accept:      application/json
```

2 Cuerpo en JSON

```
{
  "name": "Ana",
  "email": "ana@mail.com"
}
```

3 Middleware de parseo (`koa-bodyparser`)

Transforma el JSON en objeto JS accesible desde `ctx.request.body`.

EJ.04 · Enunciado

Flujo completo: `POST /api/users` → `201 Created`

Considere cabeceras, validación, middleware de parseo y formato de respuesta.

— SOLUCIÓN · PASO A PASO —

1 El cliente envía `POST /api/users` con headers

```
Content-Type: application/json
Accept:      application/json
```

2 Cuerpo en JSON

```
{
  "name": "Ana",
  "email": "ana@mail.com"
}
```

3 Middleware de parseo (`koa-bodyparser`)

Transforma el JSON en objeto JS accesible desde `ctx.request.body`.

4 Validación de datos

Campos obligatorios · formato correcto · reglas de negocio.

EJ.04 · Enunciado

Flujo completo: `POST /api/users` → `201 Created`

Considere cabeceras, validación, middleware de parseo y formato de respuesta.

— SOLUCIÓN · PASO A PASO —

1 El cliente envía `POST /api/users` con headers

```
Content-Type: application/json
Accept:      application/json
```

2 Cuerpo en JSON

```
{
  "name": "Ana",
  "email": "ana@mail.com"
}
```

3 Middleware de parseo (`koa-bodyparser`)

Transforma el JSON en objeto JS accesible desde `ctx.request.body`.

4 Validación de datos

Campos obligatorios · formato correcto · reglas de negocio.

5 Si la validación falla → `ctx.status = 400`

Se responde con Bad Request y un mensaje de error.

EJ.04 · Enunciado

Flujo completo: POST /api/users → 201 Created

Considere cabeceras, validación, middleware de parseo y formato de respuesta.

— SOLUCIÓN · PASO A PASO —

1 El cliente envía POST /api/users con headers

```
Content-Type: application/json
Accept:      application/json
```

2 Cuerpo en JSON

```
{
  "name": "Ana",
  "email": "ana@mail.com"
}
```

3 Middleware de parseo (koa-bodyparser)

Transforma el JSON en objeto JS accesible desde `ctx.request.body`.

4 Validación de datos

Campos obligatorios · formato correcto · reglas de negocio.

5 Si la validación falla → `ctx.status = 400`

Se responde con Bad Request y un mensaje de error.

6 Si es válido → crea el usuario y responde 201 Created

```
ctx.status = 201;
ctx.body = nuevoUsuario;
// Content-Type: application/json
```

[EJ.05](#) · [Enunciado](#)

Explique qué significa cada parte de este hash generado con bcrypt.

[— SOLUCIÓN](#) · [ANATOMÍA](#)

```
$2b$10$X5rT9kLmNpQrSuVwXyZ012.Yz8AbCdEfGhIjKlMnOpQrStUvWxYz
```

[EJ.05](#) · [Enunciado](#)

Explique qué significa cada parte de este hash generado con bcrypt.

[— SOLUCIÓN](#) · [ANATOMÍA](#)

```
$2b$10$X5rT9kLmNpQrSuVwXyZ012.Yz8AbCdEfGhIjKlMnOpQrStUvWxYz
```

- **\$2b\$** — Versión del algoritmo bcrypt

[EJ.05](#) · [Enunciado](#)

Explique qué significa cada parte de este hash generado con bcrypt.

[— SOLUCIÓN](#) · [ANATOMÍA](#)

```
$2b$10$X5rT9kLmNpQrSuVwXyZ012.Yz8AbCdEfGhIjKlMnOpQrStUvWxYz
```

- **\$2b\$** — Versión del algoritmo bcrypt
- **10\$** — Costo / rondas (cost factor). A mayor valor, más lento de calcular.

[EJ.05](#) · [Enunciado](#)

Explique qué significa cada parte de este hash generado con bcrypt.

[— SOLUCIÓN](#) · [ANATOMÍA](#)

```
$2b$10$X5rT9kLmNpQrSuVwXyZ012.Yz8AbCdEfGhIjKlMnOpQrStUvWxYz
```

- **\$2b\$** — Versión del algoritmo bcrypt
- **10\$** — Costo / rondas (cost factor). A mayor valor, más lento de calcular.
- **X5rT9kLmNpQrSuVwXyZ012** — Salt aleatorio único por contraseña

EJ.05 · Enunciado

Explique qué significa cada parte de este hash generado con bcrypt.

— SOLUCIÓN · ANATOMÍA

```
$2b$ 10$ X5rT9kLmNpQrSuVwXyZ012 .Yz8AbCdEfGhIjKlMnOpQrStUvWxYz
```

- **\$2b\$** — Versión del algoritmo bcrypt
- **10\$** — Costo / rondas (cost factor). A mayor valor, más lento de calcular.
- **X5rT9kLmNpQrSuVwXyZ012** — Salt aleatorio único por contraseña
- **.Yz8AbCdEfGhIjKlMnOpQrStUvWxYz** — Hash derivado de (password + salt)

[EJ.06](#) · Enunciado

¿Qué imprime `console.log`? ¿Por qué y cómo se arregla?

[CÓDIGO ORIGINAL](#)[— SOLUCIÓN](#)

```
async function obtenerDatos() {
  const response = await fetch('https://api.ejemplo.com/data');
  const data = await response.json();
  return data;
}

async function mostrar() {
  const datos = obtenerDatos();
  console.log(datos);
}

mostrar();
```


EJ.06 · Enunciado

¿Qué imprime `console.log`? ¿Por qué y cómo se arregla?

CÓDIGO ORIGINAL

```
async function obtenerDatos() {  
  const response = await fetch('https://api.ejemplo.com/data');  
  const data = await response.json();  
  return data;  
}  
  
async function mostrar() {  
  const datos = obtenerDatos();  
  console.log(datos);  
}  
  
mostrar();
```

— SOLUCIÓN

IMPRIME

`Promise { <pending> }`

EJ.06 · Enunciado

¿Qué imprime `console.log`? ¿Por qué y cómo se arregla?

CÓDIGO ORIGINAL

```
async function obtenerDatos() {
  const response = await fetch('https://api.ejemplo.com/data');
  const data = await response.json();
  return data;
}

async function mostrar() {
  const datos = obtenerDatos();
  console.log(datos);
}

mostrar();
```

— SOLUCIÓN

IMPRIME

`Promise { <pending> }`

por qué `obtenerDatos` es `async` → retorna inmediatamente una `Promise`, no el valor resuelto.

EJ.06 · Enunciado

¿Qué imprime `console.log`? ¿Por qué y cómo se arregla?

CÓDIGO ORIGINAL

```
async function obtenerDatos() {
  const response = await fetch('https://api.ejemplo.com/data');
  const data = await response.json();
  return data;
}

async function mostrar() {
  const datos = obtenerDatos();
  console.log(datos);
}

mostrar();
```

— SOLUCIÓN

IMPRIME

`Promise { <pending> }`

por qué `obtenerDatos` es `async` → retorna inmediatamente una `Promise`, no el valor resuelto.

```
async function mostrar() {
  const datos = await obtenerDatos();
  console.log(datos);
}
```

FIX · AGREGAR AWAIT

EJ.07 • Enunciado

Hay un error en el flujo asincrónico. ¿Cuál es?

MIDDLEWARE DE AUTENTICACIÓN (KOA)

[— SOLUCIÓN](#)

```
const authMiddleware = async (ctx, next) => {
  const token = ctx.headers.authorization?.split(' ')[1];
  if (!token) { ctx.status = 401; return; }
  try {
    const decoded = jwt.verify(token, secret);
    ctx.state.user = decoded;
  } catch (err) {
    ctx.status = 401; return;
  }
  next();
};

app.use(authMiddleware);
app.use(async ctx => {
  ctx.body = { user: ctx.state.user };
});
```

EJ.07 • Enunciado

Hay un error en el flujo asíncrono. ¿Cuál es?

MIDDLEWARE DE AUTENTICACIÓN (KOA)

```
const authMiddleware = async (ctx, next) => {
  const token = ctx.headers.authorization?.split(' ')[1];
  if (!token) { ctx.status = 401; return; }
  try {
    const decoded = jwt.verify(token, secret);
    ctx.state.user = decoded;
  } catch (err) {
    ctx.status = 401; return;
  }
  next();
};

app.use(authMiddleware);
app.use(async ctx => {
  ctx.body = { user: ctx.state.user };
});
```

— SOLUCIÓN —

CAMBIO

`next();` → `await next();`

EJ.07 • Enunciado

Hay un error en el flujo asíncrono. ¿Cuál es?

MIDDLEWARE DE AUTENTICACIÓN (KOA)

```
const authMiddleware = async (ctx, next) => {
  const token = ctx.headers.authorization?.split(' ')[1];
  if (!token) { ctx.status = 401; return; }
  try {
    const decoded = jwt.verify(token, secret);
    ctx.state.user = decoded;
  } catch (err) {
    ctx.status = 401; return;
  }
  next();
};

app.use(authMiddleware);
app.use(async ctx => {
  ctx.body = { user: ctx.state.user };
});
```

— SOLUCIÓN

CAMBIO

`next();` → `await next();`

por qué En Koa los middlewares son **asíncronos** y deben esperar correctamente la ejecución del siguiente middleware.

EJ.07 • Enunciado

Hay un error en el flujo asíncrono. ¿Cuál es?

MIDDLEWARE DE AUTENTICACIÓN (KOA)

```
const authMiddleware = async (ctx, next) => {
  const token = ctx.headers.authorization?.split(' ')[1];
  if (!token) { ctx.status = 401; return; }
  try {
    const decoded = jwt.verify(token, secret);
    ctx.state.user = decoded;
  } catch (err) {
    ctx.status = 401; return;
  }
  next();
};

app.use(authMiddleware);
app.use(async ctx => {
  ctx.body = { user: ctx.state.user };
});
```

— SOLUCIÓN

CAMBIO

`next();` → `await next();`

- por qué** En Koa los middlewares son **asíncronos** y deben esperar correctamente la ejecución del siguiente middleware.
- consec.** Sin `await`, se rompe el **onion model** y el middleware puede terminar antes de tiempo.

[EJ.08](#) · [Enunciado](#)

Transferencia bancaria por GET — ¿por qué es incorrecto?

PROPUESTA DEL DESARROLLADOR

```
GET http://banco.cl/api/transferir?origen=123&destino=456&monto=50000
```

— SOLUCIÓN

[EJ.08](#) • Enunciado

Transferencia bancaria por GET — ¿por qué es incorrecto?

PROPUESTA DEL DESARROLLADOR

```
GET http://banco.cl/api/transferir?origen=123&destino=456&monto=50000
```

— SOLUCIÓN

- 01 **GET debe ser seguro y sin efectos secundarios** — sirve para *obtener* información. Una transferencia modifica estado y no debe usar GET.

EJ.08 · Enunciado

Transferencia bancaria por GET — ¿por qué es incorrecto?

PROPUESTA DEL DESARROLLADOR

```
GET http://banco.cl/api/transferir?origen=123&destino=456&monto=50000
```

— SOLUCIÓN

- 01 **GET debe ser seguro y sin efectos secundarios** — sirve para *obtener* información. Una transferencia modifica estado y no debe usar GET.
- × Caché accidental (navegador / proxies).
 - × Queda en el historial del navegador.
 - × Re-ejecución involuntaria al refrescar.
 - × Datos sensibles expuestos en la URL.

EJ.08 · Enunciado

Transferencia bancaria por GET — ¿por qué es incorrecto?

PROPUESTA DEL DESARROLLADOR

```
GET http://banco.cl/api/transferir?origen=123&destino=456&monto=50000
```

— SOLUCIÓN

01 **GET debe ser seguro y sin efectos secundarios** — sirve para *obtener* información. Una transferencia modifica estado y no debe usar GET.

- × Caché accidental (navegador / proxies).
- × Queda en el historial del navegador.
- × Re-ejecución involuntaria al refrescar.
- × Datos sensibles expuestos en la URL.

capas **Confusión de capas:** TCP/IP asegura el *transporte* confiable de paquetes; HTTP define la *semántica* de aplicación.

EJ.08 • Enunciado

Transferencia bancaria por GET — ¿por qué es incorrecto?

PROPUESTA DEL DESARROLLADOR

```
GET http://banco.cl/api/transferir?origen=123&destino=456&monto=50000
```

— SOLUCIÓN

01 **GET debe ser seguro y sin efectos secundarios** — sirve para *obtener* información. Una transferencia modifica estado y no debe usar GET.

- × Caché accidental (navegador / proxies).
- × Queda en el historial del navegador.
- × Re-ejecución involuntaria al refrescar.
- × Datos sensibles expuestos en la URL.

capas **Confusión de capas:** TCP/IP asegura el *transporte* confiable de paquetes; HTTP define la *semántica* de aplicación.

ESTRUCTURA CORRECTA

```
POST /api/transferencias
```

```
Content-Type: application/json
```

```
{ "origen": 123, "destino": 456, "monto": 50000 }
```

EJ.09 · Enunciado

Prediga la salida completa e indique el orden de impresión.

— SOLUCIÓN —

```
const obj = {  
  valor: 100,  
  obtener: function() { return this.valor; },  
  obtenerFlecha: () => this.valor  
};  
  
console.log(obj.obtener());      // (a) ?  
console.log(obj.obtenerFlecha()); // (b) ?  
  
setTimeout(() => console.log("inicio"), 0); // (c) ?  
console.log("fin");                // (d) ?
```


EJ.09 · Enunciado

Prediga la salida completa e indique el orden de impresión.

```
const obj = {  
  valor: 100,  
  obtener: function() { return this.valor; },  
  obtenerFlecha: () => this.valor  
};  
  
console.log(obj.obtener());      // (a) ?  
console.log(obj.obtenerFlecha()); // (b) ?  
  
setTimeout(() => console.log("inicio"), 0); // (c) ?  
console.log("fin");                // (d) ?
```

— SOLUCIÓN

(a) `obj.obtener()` → **100**. `this` referencia a `obj`.

EJ.09 · Enunciado

Prediga la salida completa e indique el orden de impresión.

```
const obj = {  
  valor: 100,  
  obtener: function() { return this.valor; },  
  obtenerFlecha: () => this.valor  
};  
  
console.log(obj.obtener());      // (a) ?  
console.log(obj.obtenerFlecha()); // (b) ?  
  
setTimeout(() => console.log("inicio"), 0); // (c) ?  
console.log("fin");                // (d) ?
```

— SOLUCIÓN

- (a) `obj.obtener()` → **100**. `this` referencia a `obj`.
- (b) `obj.obtenerFlecha()` → **undefined**. Las flechas **no** tienen su propio `this`; lo heredan del contexto exterior.

EJ.09 · Enunciado

Prediga la salida completa e indique el orden de impresión.

```
const obj = {  
  valor: 100,  
  obtener: function() { return this.valor; },  
  obtenerFlecha: () => this.valor  
};  
  
console.log(obj.obtener());      // (a) ?  
console.log(obj.obtenerFlecha()); // (b) ?  
  
setTimeout(() => console.log("inicio"), 0); // (c) ?  
console.log("fin");                // (d) ?
```

— SOLUCIÓN

- (a) `obj.obtener()` → **100**. `this` referencia a `obj`.
- (b) `obj.obtenerFlecha()` → **undefined**. Las flechas **no tienen su propio `this`**; lo heredan del contexto exterior.
- (c)(d) `setTimeout(..., 0)` no se ejecuta inmediatamente — el callback se envía a la **Task Queue** y se ejecuta cuando el **Call Stack** queda vacío.

EJ.09 · Enunciado

Prediga la salida completa e indique el orden de impresión.

```
const obj = {
  valor: 100,
  obtener: function() { return this.valor; },
  obtenerFlecha: () => this.valor
};

console.log(obj.obtener());      // (a) ?
console.log(obj.obtenerFlecha()); // (b) ?

setTimeout(() => console.log("inicio"), 0); // (c) ?
console.log("fin");               // (d) ?
```

```
1  100
2  undefined
3  fin
4  inicio
```

— SOLUCIÓN

- (a) `obj.obtener()` → **100**. `this` referencia a `obj`.
- (b) `obj.obtenerFlecha()` → **undefined**. Las flechas **no tienen su propio `this`**; lo heredan del contexto exterior.
- (c)(d) `setTimeout(..., 0)` no se ejecuta inmediatamente — el callback se envía a la **Task Queue** y se ejecuta cuando el **Call Stack** queda vacío.